Week 5

Lab Activities

Activity 1: Using the dice sample space

$\Omega = \{1, 2, 3, 4, 5, 6\}$
give an example of:

- Two sets **A** and **B** with $\mathbf{A \subset B}$
- Two sets **A** and **B** where $\mathbf{A \subset B}$ isn't true (we could write $\mathbf{A \not\subset B}$)

Activity 2: Manipulating Events

A standard deck of cards consists of 52 cards split into four "suits": spades (♠), hearts (♥), diamonds (♦), and clubs (♣).

In each suit there are cards numbered 1 to 10 and three "face" cards: Jack (J), Queen (Q), and King (K).

Write down 5 outcomes from the sample space.

Define the following events:

- **A** is the event that the card is a heart (♥).
- **B** is the event that the card is a Jack, Queen, or King.
- **C** is the event that the card is a club (♣) and has a number that is divisible by 3.

Write out the events **A**, **B**, and **C** by listing their outcomes.

Write out the events:

- $\mathbf{A \wedge B}$,
- $\mathbf{A \vee B}$, and
- $\mathbf{A \wedge C}$,

by listing their outcomes.

Assume that a card is chosen fairly from the deck. Calculate:

- **P(A)**
- **P(B)**
- **P(A ∧ B)**
- **P(A ∨ B)**
- **P(¬C)**

Explain in words what each of these probabilities means.

Is $\mathbf{P(A \vee B) = P(A) + P(B)?}$

Bonus: Write Python code to print all 52 cards in an efficient way.

Activity 3: Generate a random integer between 1 and 100.

Activity 4:

- Generate 100 random numbers between 1 and 6.
- Add together two random numbers between 1 and 6.
- Give 100 results of adding together two random numbers between 1 and 6.
- Count how many times you got each result. (Use your pandas skills, or investigate the numpy function np.unique).

Does this correspond with your expectations (for those of you who play board games)? * Run the same simulation with 1000 then with 1,000,000 results.

Activity 5: Estimate how often we would see a total of 13 when throwing 3 six-sided die.

Activity 6: Define a Weighted Random Number Generator

When rolling two standard dice, the possible sums range from **2** to **12**, but not all sums are equally likely. For example, a sum of **7** can occur in more ways than a sum of **2** or **12**.

In this activity, use the **choice** (or random.choices) method to create a **weighted random number generator** that randomly selects a "sum of two dice" according to the **correct probabilities**.

Your generator should ensure that numbers like **7** appear more frequently than **2** or **12**, reflecting their true likelihood in real dice rolls.

Activity 7: Using the definition of conditional probability determine the probability of a fair six-sided dice showing a prime number given that it shows a number greater than two.

Activity 8: Suppose we are picking an integer (whole number) uniformly at random between 1 and 24.

Let events **A**, **B**, and **C** be defined in the following way:

$$A = \text{The integer picked is divisible by 3}$$
$$B = \text{The integer picked is divisible by 4}$$
$$C = \text{The integer picked is divisible by 6}$$

Calculate:

- **A**
- **B**
- **A ∩ B**

- **A ∪ B**

Calculate:

- **P(B | C)**
- **P(C | B)**

Question: Is any pair of events **A**, **B**, **C** independent?

Activity 9 – Naive Bayes and Spam Detection

We want to find out how likely an email is to be Spam if it contains the words “lottery” and “urgent”.

Using the Naive Bayes assumption (that the words appear independently given the class), calculate:

P(Spam | “lottery” ∧ “urgent”)

Given probabilities:

Event	Probability
P(Spam)	0.4
P(Not Spam)	0.6
P("lottery"	Spam)
P("urgent"	Spam)
P("lottery"	Not Spam)
P("urgent"	Not Spam)

Tasks:

1. Calculate the probability that both words appear in a Spam email.
   (Hint: multiply P("lottery" | Spam) × P("urgent" | Spam))
2. Calculate the probability that both words appear in a Not Spam email.
   (Hint: multiply P("lottery" | Not Spam) × P("urgent" | Not Spam))
3. Use Bayes’ theorem to find:
   P(Spam | “lottery” ∧ “urgent”)
4. Interpret your result in plain language:
   How much more likely is the email to be spam when both words appear?